

LAPORAN PRAKTIKUM

STRUKTUR DATA

“QUEUE”



Disusun oleh:

Ndaru Pradana Gatot Suseno

2411532007

Dosen Pengampu:

Dr. Wahyudi, S.T, M.T

Asisten Praktikum:

Jovantri Immanuel Gulo

**DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS**

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat-Nya Laporan Praktikum Struktur Data 4 dapat diselesaikan. Laporan ini menjadi dokumentasi kegiatan praktikum sekaligus media untuk memahami struktur data queue dalam bahasa Java.

Praktikum keempat membahas pola First In First Out melalui implementasi queue berbasis array dan LinkedList. Program juga menguji iterasi elemen serta pembalikan queue dengan bantuan stack. Pada laporan tugas, konsep queue diterapkan pada sistem antrian loket yang dapat menambah pelanggan, melayani pelanggan, menampilkan antrean, dan membalik urutan antrean.

Penulis berterima kasih kepada dosen pengampu dan asisten praktikum atas bimbingan yang diberikan. Pembahasan disusun berdasarkan source code package pekan4_2411532007 dan hasil eksekusi secara langsung. Kritik serta saran yang membangun diharapkan untuk penyempurnaan laporan selanjutnya.

Padang, 2026

Ndaru Pradana G.S

DAFTAR ISI

Contents

KATA PENGANTAR	2
DAFTAR ISI.....	3
BAB I PENDAHULUAN.....	4
Latar Belakang	4
Tujuan Praktikum.....	4
Manfaat Praktikum.....	4
BAB II PEMBAHASAN	5
Dasar Teori.....	5
Program QueueArray_2411532007 dan Driver	6
Program QueueLinkedList_2411532007 dan IterasiQueue_2411532007.....	8
BAB III KESIMPULAN.....	10
DAFTAR PUSTAKA	11
LAPORAN TUGAS	12
Tujuan Tugas.....	12
Class AntrianLoket_2411532007	13
Hasil Eksekusi Laporan Tugas.....	15
Analisis Hasil	15
Kesimpulan Laporan Tugas	15
Saran.....	15

BAB I

PENDAHULUAN

Latar Belakang

Queue merupakan struktur data linear dengan proses penambahan pada bagian rear dan penghapusan pada bagian front. Elemen yang pertama masuk akan menjadi elemen pertama yang keluar, sehingga mekanismenya disebut First In First Out. Queue banyak digunakan pada sistem pelayanan, penjadwalan proses, buffer, dan pengiriman pesan.

Implementasi queue dapat menggunakan array dengan indeks front dan rear atau menggunakan LinkedList yang mendukung penambahan dan penghapusan secara dinamis. Praktikum ini memperlihatkan operasi enqueue, dequeue, front, rear, iterasi, serta pembalikan urutan dengan memindahkan elemen ke stack.

Tujuan Praktikum

1. Memahami konsep FIFO, front, dan rear pada queue.
2. Menerapkan enqueue, dequeue, peek/front, rear, dan display.
3. Mengimplementasikan queue dengan array dan LinkedList.
4. Melakukan iterasi dan reverse terhadap isi queue.
5. Menerapkan queue pada sistem antrian loket.

Manfaat Praktikum

Praktikum ini membantu memahami pengelolaan data berdasarkan urutan kedatangan, validasi kondisi penuh dan kosong, serta perbedaan implementasi queue statis dan dinamis. Konsep tersebut relevan untuk sistem pelayanan, antrean pekerjaan, dan pemrosesan data berurutan.

BAB II

PEMBAHASAN

Dasar Teori

Operasi enqueue menambahkan elemen pada rear, sedangkan dequeue mengambil elemen dari front. Kondisi kosong perlu diperiksa sebelum penghapusan dan kondisi penuh perlu diperiksa pada implementasi array berkapasitas tetap.

LinkedList dapat digunakan sebagai implementasi Queue melalui interface `java.util.Queue`. Pembalikan urutan queue dapat dilakukan dengan memindahkan seluruh elemen ke stack, kemudian memasukkannya kembali sehingga urutan FIFO berubah berdasarkan sifat LIFO stack.

Program QueueArray_2411532007 dan Driver

QueueArray_2411532007 menggunakan array, capacity, size, front, dan rear untuk mengelola antrian secara sirkular. QueueArrayDriver_2411532007 menguji empat enqueue, pemeriksaan front dan rear, display, serta dequeue.

Source Code QueueArray_2411532007

```

1 package pekan4_2411532007;
2
3 import java.util.Queue;
4
5 public class QueueArray_2411532007 {
6     int front_2007, rear_2007, size_2007;
7     int capacity_2007;
8     int array_2007[];
9
10    public QueueArray_2411532007(int capacity) {
11        this.capacity_2007 = capacity;
12        front_2007 = this.size_2007 = 0;
13        rear_2007 = capacity - 1;
14        array_2007 = new int [this.capacity_2007];
15    }
16
17    boolean isFull_2007(QueueArray_2411532007 queue) {
18        return (queue.size_2007 == queue.capacity_2007);
19    }
20
21    boolean isEmpty_2007 (QueueArray_2411532007 queue) {
22        return (queue.size_2007 == 0);
23    }
24
25    void enqueue_2007(int item_2007) {
26        if (isFull_2007(this)) {
27            return;
28        }
29        this.rear_2007 = (this.rear_2007 + 1) % this.capacity_2007;
30        this.array_2007[this.rear_2007] = item_2007;
31        this.size_2007 = this.size_2007 + 1;
32        System.out.println(item_2007 + " enqueued to queue");
33    }
34
35    int dequeue_2007() {
36        if (isEmpty_2007(this)) {
37            return Integer.MIN_VALUE;
38        }
39        int item_2007 = this.array_2007[this.front_2007];
40        this.front_2007 = (this.front_2007 + 1) % this.capacity_2007;
41        this.size_2007 = this.size_2007 - 1;
42        return item_2007;
43    }
44
45    int front_2007() {
46        if (isEmpty_2007(this)) {
47            return Integer.MIN_VALUE;
48        }
49        return this.array_2007[this.front_2007];
50    }
51
52    int rear_2007() {
53        if (isEmpty_2007(this)) {
54            return Integer.MIN_VALUE;
55        }
56        return this.array_2007[this.rear_2007];
57    }
58
59    void display_2007() {
60        int i;
61        if (front_2007 == rear_2007) {
62            System.out.printf("\nantrian kosong\n");
63            return;
64        }
65        for ( i = front_2007; i < rear_2007; i++) {
66            System.out.printf(" %d <--", array_2007[i]);
67        }
68        return;
69    }
70 }
71

```

Source Code QueueArrayDriver_2411532007

```

1 package pekan4_2411532007;
2
3 public class QueueArrayDriver_2411532007 {
4     public static void main ( String [] args) {
5         QueueArray_2411532007 queue_2007 = new QueueArray_2411532007(1000);
6
7         queue_2007.enqueue_2007(10);
8         queue_2007.enqueue_2007(20);
9         queue_2007.enqueue_2007(30);
10        queue_2007.enqueue_2007(40);
11
12        System.out.println("item di depan " + queue_2007.front_2007());
13        System.out.println("item di paling belakang " + queue_2007.rear_2007());
14        System.out.println("tampilkan queue");
15
16        queue_2007.display_2007();
17
18        System.out.println();
19        System.out.println(queue_2007.dequeue_2007() + " dihapus dari queue");
20        System.out.println("item di depan : " + queue_2007.front_2007());
21        System.out.println("item di belakang : " + queue_2007.rear_2007());
22
23        System.out.println("tampilkan queue setelah dihapus");
24        queue_2007.display_2007();
25    }
26 }
27

```

Penjelasan Operasi

1. enqueue_2007() menggeser rear secara modulo kapasitas dan menambah size.
2. dequeue_2007() mengambil data front lalu menggeser front.
3. front_2007() dan rear_2007() membaca elemen terdepan dan terakhir.
4. display_2007() menampilkan elemen antrean pada array.

Hasil Eksekusi

```

<terminated> QueueArrayDriver_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justi.openjdk.hotspot.jre.full.win32.x86_64.23.0.1.v20241024-1700\jre\bin\javaw.exe
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
item di depan 10
item di paling belakang 40
tampilkan queue
10 <-- 20 <-- 30 <--
10 dihapus dari queue
item di depan :20
item di belakang :40
tampilkan queue setelah dihapus
20 <-- 30 <--

```

Analisis

Nilai 10, 20, 30, dan 40 masuk secara FIFO. Front bernilai 10 dan rear bernilai 40. Setelah dequeue, nilai 10 keluar dan front berubah menjadi 20.

Method display menggunakan batas $i < \text{rear}$ sehingga elemen pada indeks rear tidak ikut dicetak. Akibatnya nilai 40 tidak terlihat pada baris tampilan walaupun masih tersimpan sebagai elemen terakhir.

Program QueueLinkedList_2411532007 dan IterasiQueue_2411532007

QueueLinkedList menggunakan Queue<Integer> dengan LinkedList, sedangkan IterasiQueue menggunakan Iterator untuk menelusuri seluruh elemen String tanpa menghapusnya.

Source Code QueueLinkedList_2411532007

```
1 package pekan4_2411532007;
2
3 import java.util.Queue;
4
5
6 public class QueueLinkedList_2411532007 {
7     public static void main (String [] args) {
8         Queue<Integer> q_2007 = new LinkedList<>();
9
10        for ( int i = 0; i < 6; i++)
11            q_2007.add(i);
12
13        System.out.println("Element Antrian " + q_2007);
14
15        int hapus_2007 = q_2007.remove();
16
17        System.out.println("Hapus Elemen = " + hapus_2007);
18        System.out.println(q_2007);
19
20        int depan_2007 = q_2007.peek();
21        System.out.println("Kepala antrian = " + depan_2007);
22
23        int banyak_2007 = q_2007.size();
24        System.out.println("Size antrian = " + banyak_2007);
25    }
26 }
27
```

Source Code IterasiQueue_2411532007

```
1 package pekan4_2411532007;
2
3 import java.util.Iterator;
4
5
6 public class IterasiQueue_2411532007 {
7     public static void main (String [] args) {
8         Queue<String> q_2007 = new LinkedList<>();
9
10        q_2007.add("Pratikum");
11        q_2007.add("struktur");
12        q_2007.add("Data");
13        q_2007.add("Dan");
14        q_2007.add("Algoritma");
15
16        Iterator<String> iterator_2007 = q_2007.iterator();
17
18        while(iterator_2007.hasNext()) {
19            System.out.print(iterator_2007.next() + " ");
20        }
21    }
22 }
23
```

Hasil Eksekusi

```
<terminated> QueueLinkedList_2411532007 [Java Application] C:\Users\ndaru\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_23.0.1.v20241024-1700\jre\bin
Element Antrian [0, 1, 2, 3, 4, 5]
Hapus Elemen = 0
[1, 2, 3, 4, 5]
Kepala antrian = 1
Size antrian = 5
```

Analisis

Queue mula-mula berisi nilai 0 sampai 5. remove() mengeluarkan nilai 0 karena berada di front, kemudian peek() menghasilkan 1 dan size() menunjukkan lima elemen tersisa.

Iterator pada IterasiQueue membaca elemen “Pratikum struktur Data Dan Algoritma” sesuai urutan masuk tanpa mengubah isi queue.

Program ReverseData_2411532007

Program ReverseData membalik urutan queue dengan stack. Seluruh elemen queue dihapus menuju stack, kemudian dikeluarkan dari stack untuk dimasukkan kembali ke queue.

Source Code ReverseData_2411532007

```

1 package pekan4_2411532007;
2 import java.util.Stack;
3
4
5 public class ReverseData_2411532007 {
6     public static void main (String []args) {
7         Queue<Integer> q_2007 = new LinkedList<Integer>();
8
9         q_2007.add(1);
10        q_2007.add(2);
11        q_2007.add(3);
12
13        System.out.println("sebelum reverse " + q_2007 );
14
15        Stack<Integer> s_2007 = new Stack<Integer>();
16
17        while(!q_2007.isEmpty()) {
18            s_2007.push(q_2007.remove());
19        }
20
21        while(!s_2007.isEmpty()) {
22            q_2007.add(s_2007.pop());
23        }
24        System.out.println("Sesudah reverse= " + q_2007);
25    }
26 }
27
28
29

```

Hasil Eksekusi

```

sebelum reverse [1, 2, 3]
sesudah reverse= [3, 2, 1]

```

Analisis

Queue awal [1, 2, 3] dipindahkan ke stack sehingga 3 berada di top. Ketika stack dikosongkan kembali ke queue, urutannya menjadi [3, 2, 1].

Proses ini mempunyai kompleksitas waktu linear karena setiap elemen dipindahkan dua kali, yaitu dari queue ke stack dan dari stack ke queue.

BAB III

KESIMPULAN

Praktikum 4 berhasil menerapkan queue berbasis array dan LinkedList serta memperlihatkan operasi enqueue, dequeue, front, rear, iterasi, dan reverse. Setiap operasi mengikuti prinsip FIFO.

Implementasi array membutuhkan pengelolaan indeks dan kapasitas secara eksplisit, sedangkan LinkedList menyediakan pengelolaan ukuran secara dinamis. Queue juga dapat dikombinasikan dengan stack untuk mengubah urutan data.

DAFTAR PUSTAKA

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data Structures and Algorithms in Java (6th ed.). Wiley.

Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.

Sierra, K., Bates, B., & Gee, T. (2022). Head First Java (3rd ed.). O'Reilly Media.

Oracle. (2024). Java Platform, Standard Edition API Specification. Oracle Corporation.

LAPORAN TUGAS

Laporan tugas Praktikum 4 membangun sistem Antrian Locket. Pada source repository, struktur queue dan driver interaktif berada dalam satu file `AntrianLocket_2411532007.java`; tidak terdapat file terpisah bernama `AntrianLocketDriver_2411532007`. Oleh karena itu, bagian driver dibahas melalui method `main` pada class tersebut.

Tujuan Tugas

1. Membuat queue pelanggan berbasis array String.
2. Menambahkan pelanggan melalui `enqueue`.
3. Melayani pelanggan terdepan melalui `dequeue`.
4. Menampilkan dan membalik urutan antrian.
5. Menguji seluruh operasi melalui menu driver interaktif.

Class AntrianLoket_2411532007

Class AntrianLoket_2411532007 memiliki atribut front, rear, kapasitas maksimum, dan array String. Method enqueue, dequeue, display, dan reverse membentuk operasi utama queue loket.

Source Code

```

1 package pekan4_2411532007;
2 import java.util.Scanner;
3
4 public class AntrianLoket_2411532007 {
5
6     int front_2007;
7     int rear_2007;
8     int max_2007;
9     String queue_2007[];
10
11     public AntrianLoket_2411532007(int kapasitas) {
12         this.max_2007 = kapasitas;
13         this.queue_2007 = new String[max_2007];
14         this.front_2007 = 0;
15         this.rear_2007 = -1;
16     }
17
18
19     public boolean isEmpty() {
20         return rear_2007 < front_2007;
21     }
22
23     public boolean isFull() {
24         return rear_2007 == max_2007 - 1;
25     }
26
27     public void enqueue(String data) {
28         if (isFull()) {
29             System.out.println("Antrian penuh!");
30             return;
31         }
32         rear_2007++;
33         queue_2007[rear_2007] = data;
34         System.out.println("Data berhasil ditambahkan ke antrian");
35     }
36
37
38     public void dequeue() {
39         if (isEmpty()) {
40             System.out.println("Antrian kosong!");
41             return;
42         }
43         System.out.println(queue_2007[front_2007] + " telah dilayani");
44         front_2007++;
45
46         if (isEmpty()) {
47             front_2007 = 0;
48             rear_2007 = -1;
49         }
50     }
51
52     public void display() {
53         if (isEmpty()) {
54             System.out.println("Antrian kosong.");
55             return;
56         }
57         System.out.println("Isi antrian:");
58         for (int i = front_2007; i <= rear_2007; i++) {
59             System.out.println((i - front_2007 + 1) + ". " + queue_2007[i]);
60         }
61     }
62
63     public void reverse() {
64         if (isEmpty()) {
65             System.out.println("Antrian kosong.");
66             return;
67         }
68         int left = front_2007;
69         int right = rear_2007;
70
71         while (left < right) {
72             String temp = queue_2007[left];
73             queue_2007[left] = queue_2007[right];
74             queue_2007[right] = temp;
75             left++;
76             right--;
77         }
78     }
79
80
81     public static void main(String[] args) {
82         Scanner scanner = new Scanner(System.in);
83         AntrianLoket_2411532007 antrian = new AntrianLoket_2411532007(10);
84
85         int pilihan_2007 = 0;

```

```

85     int pilihan_2007 = 0;
86
87     do {
88         System.out.println("\n== PROGRAM ANTRIAN LOKET ==");
89         System.out.println("1. Tambah Antrian");
90         System.out.println("2. Hapus Antrian");
91         System.out.println("3. Tampilkan Antrian");
92         System.out.println("4. Reverse");
93         System.out.println("5. Keluar");
94         System.out.print("Pilih menu: ");
95
96         if (scanner.hasNextInt()) {
97             pilihan_2007 = scanner.nextInt();
98             scanner.nextLine();
99         } else {
100            System.out.println("Input tidak valid. Masukkan angka.");
101            scanner.nextLine();
102            continue;
103        }
104
105        switch (pilihan_2007) {
106            case 1:
107                System.out.print("Masukkan nama pelanggan: ");
108                String nama = scanner.nextLine();
109                antrian.enqueue(nama);
110                break;
111            case 2:
112                antrian.dequeue();
113                break;
114            case 3:
115                antrian.display();
116                break;
117            case 4:
118                antrian.reverse();
119                break;
120            case 5:
121                System.out.println("Program selesai");
122                break;
123            default:
124                System.out.println("Pilihan tidak valid.");
125                break;
126        }
127    } while (pilihan_2007 != 5);
128
129    scanner.close();
130 }
131 }

```

Penjelasan

1. isEmpty() memeriksa apakah rear lebih kecil daripada front.
2. isFull() memeriksa apakah rear telah mencapai indeks kapasitas terakhir.
3. enqueue() menambahkan nama pelanggan pada rear.
4. dequeue() melayani pelanggan pada front dan mereset indeks ketika antrean kembali kosong.
5. reverse() menukar elemen dari kedua ujung antrean.

Hasil Eksekusi Laporan Tugas

```

== PROGRAM ANTRIAN LOKET ==
. Tambah Antrian
. Hapus Antrian
. Tampilkan Antrian
. Reverse
. Keluar
ilih menu: 1
asukkan nama pelanggan: andi
ata berhasil ditambahkan ke antrian

== PROGRAM ANTRIAN LOKET ==
. Tambah Antrian
. Hapus Antrian
. Tampilkan Antrian
. Reverse
. Keluar
ilih menu: 1
asukkan nama pelanggan: budi
ata berhasil ditambahkan ke antrian

== PROGRAM ANTRIAN LOKET ==
. Tambah Antrian
. Hapus Antrian
. Tampilkan Antrian
. Reverse
. Keluar
ilih menu: 4

== PROGRAM ANTRIAN LOKET ==
. Tambah Antrian
. Hapus Antrian
. Tampilkan Antrian
. Reverse
. Keluar
ilih menu: 3
si antrian:
. budi
. andi

```

Analisis Hasil

Andi dan Budi ditambahkan sehingga urutan awal adalah Andi lalu Budi. Setelah reverse, urutannya berubah menjadi Budi lalu Andi. Operasi dequeue kemudian melayani Budi karena ia berada pada front setelah pembalikan.

Penampilan terakhir hanya memuat Andi. Hasil tersebut menunjukkan bahwa enqueue, display, reverse, dan dequeue telah berjalan sesuai kondisi antrian yang sedang aktif.

Kesimpulan Laporan Tugas

Program AntrianLoket_2411532007 berhasil menerapkan queue pelanggan berbasis array dan menu interaktif. Walaupun class driver tidak dipisahkan ke file tersendiri, method main menjalankan fungsi driver dengan baik.

Saran

Program dapat dikembangkan menjadi circular queue agar ruang indeks yang telah dilewati front dapat digunakan kembali, menambahkan nomor antrian otomatis, waktu kedatangan, loket pelayanan ganda, dan validasi input yang lebih lengkap.